

Machine Learning Development Life Cycle

Module 9 Study Notes • The MLDLC Workflow

1. Introduction to MLDLC

Similar to the Software Development Life Cycle (SDLC) in traditional software engineering, the **Machine Learning Development Life Cycle (MLDLC)** is a standardized set of guidelines and stages used to build end-to-end Machine Learning products. It shifts the focus from simply "training an algorithm" to actually deploying a reliable software product for end-users.

2. The 9 Stages of MLDLC

A professional ML project typically flows through these nine distinct phases:

1 Framing the Problem

Before writing any code, the objective must be clearly defined. What is the business problem? Who are the end-users? What is the budget? Is this a supervised or unsupervised problem? Will it run in batch or online mode? This phase defines the architecture and scope of the entire project.

2 Data Gathering

Data rarely comes in a clean CSV file in the real world. Depending on the architecture, data is collected via:

- API integrations.
- Web Scraping (e.g., using Python to extract data directly from HTML pages).
- Extracting from secure corporate databases or data warehouses (using ETL - Extract, Transform, Load processes).
- Big Data clusters (like Hadoop/Spark).

3 Data Preprocessing

Raw data is almost always "dirty." Preprocessing ensures the ML algorithm can safely consume it. Typical tasks include:

- Removing duplicates and handling missing values.
- Removing extreme outliers that could skew the model.

- **Scaling:** Converting values that vary wildly (e.g., millions vs. decimals) into a standardized range so algorithms calculating mathematical distances don't break.

4 Exploratory Data Analysis (EDA)

The phase where engineers investigate the data to find underlying patterns, trends, and relationships between inputs and outputs. This relies heavily on data visualization (graphs, scatter plots) and statistical analysis (Univariate, Bivariate, and Multivariate analysis).

EDA also involves identifying if the dataset is imbalanced (e.g., having 10,000 photos of cats but only 10 photos of dogs) and determining strategies to fix it.

5 Feature Engineering & Selection

- **Feature Engineering:** Creating new, highly relevant input columns from existing ones to make learning easier for the algorithm (e.g., combining "Number of Rooms" and "Number of Bathrooms" into a single "Square Footage" feature).
- **Feature Selection:** Removing irrelevant columns. Dropping unnecessary features speeds up training time and prevents "Garbage In, Garbage Out."

6 Model Training, Evaluation & Selection

You rarely train just one algorithm. Engineers pass the data through multiple algorithms (Linear Regression, Random Forest, Neural Networks, etc.) to see which performs best.

- **Evaluation:** Using mathematical metrics (like accuracy, mean squared error, or silhouette scores) to grade each model.
- **Hyperparameter Tuning:** Tweaking the internal "settings" of the chosen model to squeeze out maximum performance.
- **Ensemble Learning:** Sometimes, engineers combine the predictions of multiple weak models to create one highly accurate, robust "super-model."

7 Model Deployment

Converting the trained model into a usable piece of software. Typically, the model is exported as a binary file (e.g., a Pickle file in Python). This file is wrapped in an API (Application Programming Interface) and hosted on a cloud server (AWS, Heroku, GCP). Users interact with a website or app, which sends data to the API, queries the model, and returns the prediction.

8 Testing

Before launching to the public, the deployed model is tested rigorously. A common approach is **Beta Testing** or **A/B Testing**, where the new model is released to a small, trusted subset of live users. Their feedback and the model's live performance determine if the model is ready for full release or if it needs to be rolled back to an earlier stage.

9 Optimization & Operations (MLOps)

Once live, the system must be maintained. This includes:

- Setting up automatic backups for data and models.
- Implementing Load Balancing so the server doesn't crash if millions of users query the model simultaneously.
- **Combatting Model Rot:** Real-world data evolves (e.g., people start wearing masks, which breaks older facial recognition models). An automated retraining schedule must be established to keep the model updated with fresh data.